

PANTHERSCHOOLS · SALESFORCE DEVOPS SERIES

Git Cheat Sheet

Complete Command Reference · All Workflows · Best Practices

SETUP & CONFIG

BRANCH

COMMIT

MERGE & REBASE

REMOTE

STASH

LOG & DIFF

UNDO & RESET

150+
COMMANDS

8
SECTIONS

6
WORKFLOWS

A4
PRINT READY

PANTHERSCHOOLS.COM · GIT CHEATSHEET · SALESFORCE DEVOPS SERIES

Working Dir untracked / modified	Staging Area git add -	Local Repo git commit -	Remote Repo git push -	Other Devs git pull / fetch
--	----------------------------------	-----------------------------------	----------------------------------	---------------------------------------

 **SETUP & CONFIG**

IDENTITY

`git config --global user.name "Name"`

Set global username

`git config --global user.email "e@x.com"`

Set global email

`git config --local user.name "Name"`

Set per-repo username

EDITOR & DEFAULTS

`git config --global core.editor "code --wait"`

Use VS Code as editor

`git config --global init.defaultBranch main`

Default branch name

`git config --global core.autocrlf input`

Line ending handling

`git config --global pull.rebase true`

Rebase on pull by default

INSPECT CONFIG

`git config --list`

List all settings

`git config --list --show-origin`

Show settings + source file

`git config user.name`

Show a single value

 **INIT & CLONE**

START A REPO

`git init`

Initialize repo in current dir

`git init <folder>`

Create new repo in folder

`git init --bare repo.git`

Create bare (server) repo

CLONE

`git clone <url>`

Clone remote repo

`git clone <url> <folder>`

Clone into specific folder

`git clone --depth 1 <url>`

Shallow clone (latest only)

`git clone --branch <b> <url>`

Clone specific branch

`git clone --single-branch <url>`

Clone only one branch

STATUS & INFO

`git status`

Show working tree status

`git status -s`

Short / compact status

`git status --porcelain`

Machine-readable status

 **STAGING AREA (INDEX)**

ADD FILES

`git add <file>`

Stage a specific file

`git add .`

Stage all changes in current dir

`git add -A`

Stage all (including deletions)

`git add -p`

Interactive partial stage (hunks)

`git add -i`

Interactive staging menu

`git add *.cls`

Stage by glob pattern

UNSTAGE & REMOVE

`git restore --staged <file>`

Unstage without discarding

`git restore <file>`

Discard working dir changes

`git rm <file>`

Remove file from repo + disk

`git rm --cached <file>`

Remove from repo, keep on disk

`git mv <old> <new>`

Rename / move a tracked file

.GITIGNORE

`echo ".env" >> .gitignore`

Ignore a file pattern

`git check-ignore -v <file>`

Explain why file is ignored

 **COMMIT**

`git commit -m "message"`

Commit with inline message

`git commit`

Open editor for commit message

`git commit -am "msg"`

Stage all tracked + commit

`git commit --amend`

Edit last commit (msg or files)

`git commit --amend --no-edit`

Add to last commit, keep msg

`git commit --allow-empty -m "x"`

Commit with no file changes

CONVENTIONAL COMMITS

`feat:`

New feature

`fix:`

Bug fix

`docs: refactor: test: chore:`

Other types

 **BRANCH**

LIST & CREATE

`git branch`

List local branches

`git branch -a`

List all (local + remote)

`git branch -vv`

List with upstream tracking

`git branch <name>`

Create new branch

`git checkout -b <name>`

Create & switch to branch

`git switch -c <name>`

Create & switch (modern)

SWITCH, RENAME, DELETE

`git switch <name>`

Switch to existing branch

`git switch -`

Switch to previous branch

`git branch -m <new>`

Rename current branch

`git branch -d <name>`

Delete merged branch

`git branch -D <name>`

Force delete branch

`git branch -u origin/<b>`

Set upstream tracking

 **REMOTE**

MANAGE REMOTES

`git remote -v`

List remotes with URLs

`git remote add origin <url>`

Add remote

`git remote set-url origin <url>`

Update remote URL

`git remote remove <name>`

Remove a remote

`git remote show origin`

Inspect remote details

FETCH · PULL · PUSH

`git fetch origin`

Download (don't merge)

`git fetch --all --prune`

Fetch all + remove stale refs

`git pull origin main`

Fetch + merge

`git pull --rebase origin main`

Fetch + rebase

`git push origin <branch>`

Push branch to remote

`git push -u origin <branch>`

Push + set upstream

`git push --force-with-lease`

Safe force push

`git push origin --delete <b>`

Delete remote branch

 LOG & HISTORY

VIEW HISTORY

git log * Full commit log

git log --oneline * One line per commit

git log --oneline --graph * Visual branch graph

git log --oneline --all * All branches in graph

git log -n 10 * Last 10 commits only

git log --author="Name" * Filter by author

git log --since="2 weeks ago" * Filter by date range

git log --grep="keyword" * Search commit messages

git log -S "string" * Find when string was added/removed

git log -- <file> * History for specific file

git log --follow <file> * History including renames

INSPECT A COMMIT

git show <hash> * Show commit diff + metadata

git show <hash>:<file> * Show file at a commit

git blame <file> * Who changed each line

git shortlog -s -n * Commit count by author

 DIFF

COMPARE CHANGES

git diff * Unstaged vs last commit

git diff --staged * Staged vs last commit

git diff HEAD * All changes vs last commit

git diff <hash1> <hash2> * Between two commits

git diff main..feature * Between two branches (2-dot)

git diff main...feature * Changes since branch diverged (3-dot)

git diff --stat * Summary of changed files

git diff --name-only * Only filenames

git diff -- <file> * Diff for specific file only

git diff --word-diff * Highlight changed words

INSPECT OBJECTS

git cat-file -p <hash> * Print contents of any object

git ls-tree HEAD * List files in current tree

git ls-files * Show all tracked files

git rev-parse HEAD * Full SHA of HEAD

 TAGS

CREATE TAGS

git tag v1.0.0 * Lightweight tag on HEAD

git tag -a v1.0.0 -m "Release" * Annotated tag with message

git tag -a v1.0.0 <hash> * Tag a specific commit

LIST & PUSH

git tag * List all tags

git tag -l "v1.*" * Filter tags by pattern

git show v1.0.0 * Show tag details

git push origin v1.0.0 * Push a single tag

git push origin --tags * Push all tags

git tag -d v1.0.0 * Delete local tag

git push origin --delete v1.0.0 * Delete remote tag

CHECKOUT A TAG

git checkout v1.0.0 * Detached HEAD at tag

git checkout -b <branch> v1.0.0 * Branch from tag

 MERGE

BASIC MERGE

git merge <branch> * Merge into current branch

git merge --no-ff <branch> * Force merge commit

git merge --squash <branch> * Squash all into one

git merge --abort * Cancel in-progress merge

git merge --continue * Resume after conflict fix

CONFLICT RESOLUTION

git status * Find conflicted files

git checkout --ours <f> * Accept current branch version

git checkout --theirs <f> * Accept incoming version

git mergetool * Open visual merge tool

UNDO & PREVIEW

git diff main...feature * Preview what will merge

git log main..feature * Commits not in main

git merge-base main feat * Find common ancestor

git reset --hard ORIG_HEAD * Undo a merge (local)

git revert -m 1 <hash> * Revert a merge commit safely

STRATEGIES

git merge -X ours <branch> * Auto-accept our side on conflict

git merge -X theirs <branch> * Auto-accept their side

git merge -s octopus b1 b2 * Merge multiple branches

 REBASE

BASIC REBASE

git rebase main * Rebase current branch onto main

git rebase --abort * Cancel in-progress rebase

git rebase --continue * Continue after conflict fix

git rebase --skip * Skip current conflicting commit

INTERACTIVE COMMANDS

pick	Keep commit as-is
reword	Edit commit message
edit	Pause and amend commit
squash	Combine with previous commit
fixup	Squash, discard message
drop	Delete this commit
exec	Run shell command after

INTERACTIVE REBASE

git rebase -i HEAD~3 * Rewrite last 3 commits

git rebase -i <hash> * Edit from a specific commit

⚠ Never rebase commits already pushed to shared branches. It rewrites history and breaks teammates' repos.

 **UNDO & RESET**

GIT REVERT (SAFE)

`git revert HEAD` * Undo last commit (new commit)

`git revert <hash>` * Revert a specific commit

`git revert HEAD~3..HEAD` * Revert a range

`git revert --no-commit <h>` * Stage revert, don't commit

GIT RESET (REWRITES HISTORY)

`git reset --soft HEAD~1` * Undo commit, keep staged

`git reset --mixed HEAD~1` * Undo commit, keep unstaged

`git reset --hard HEAD~1` * Undo commit, discard all

`git reset --hard ORIG_HEAD` * Undo last merge

RESTORE & CLEAN

`git restore <file>` * Discard working dir changes

`git restore --staged <file>` * Unstage a file

`git clean -fd` * Remove untracked files+dirs

`git clean -n` * Dry-run: show what'd be removed

REFLOG (RECOVERY)

`git reflog` * History of HEAD movements

`git checkout HEAD@{2}` * Restore to 2 moves ago

 **STASH**

SAVE TO STASH

`git stash` * Stash all tracked changes

`git stash push -m "label"` * Stash with a descriptive name

`git stash -u` * Include untracked files

`git stash -a` * Include ignored files too

`git stash push -- <file>` * Stash a specific file only

LIST & INSPECT

`git stash list` * Show all stash entries

`git stash show stash@{0}` * Summary of stash contents

`git stash show -p stash@{0}` * Full diff of a stash

RESTORE & REMOVE

`git stash pop` * Apply latest + remove from stack

`git stash apply stash@{1}` * Apply specific stash, keep it

`git stash drop stash@{1}` * Delete a stash entry

`git stash clear` * Remove ALL stash entries

`git stash branch <name>` * Create branch from stash

 **CHERRY-PICK**

`git cherry-pick <hash>` * Apply one commit

`git cherry-pick A^..B` * Apply range A to B (inclusive)

`git cherry-pick --no-commit <h>` * Stage without committing

`git cherry-pick -x <hash>` * Append original hash to msg

`git cherry-pick --abort` * Cancel cherry-pick

`git cherry-pick --continue` * Resume after conflict fix

 **WORKTREE**

`git worktree add ../path <b>` * New dir for a branch

`git worktree add -b <new> ../p` * Create worktree + new branch

`git worktree list` * List all worktrees

`git worktree remove ../path` * Remove a linked worktree

`git worktree prune` * Clean stale references

 **BISECT (BUG HUNT)**

`git bisect start` * Start binary search

`git bisect bad` * Mark current commit as bad

`git bisect good <hash>` * Mark known-good commit

`git bisect good / bad` * Navigate until bug found

`git bisect reset` * Exit bisect mode

`git bisect run ./test.sh` * Auto-run test script

 **SUBMODULES**

`git submodule add <url> path` * Add a submodule

`git submodule init` * Initialize submodules

`git submodule update --init` * Clone + init submodules

`git submodule update --remote` * Pull latest in submodules

`git submodule foreach git pull` * Pull in all submodules

`git clone --recurse-submodules` * Clone with submodules

`git submodule status` * Show submodule state

 **HOOKS & ALIASES**

COMMON HOOKS (IN .GIT/HOOKS/)

<code>pre-commit</code>	Run tests/lint before commit
<code>commit-msg</code>	Validate commit message format
<code>pre-push</code>	Run checks before push
<code>post-merge</code>	Run actions after merge

USEFUL ALIASES

`git config --global alias.st status`

`git config --global alias.co checkout`

`git config --global alias.br branch`

`git config --global alias.lg "log --oneline --graph --all"`

`git config --global alias.undo "reset --soft HEAD~1"`

WF / Common Workflows

 FEATURE BRANCH WORKFLOW

```
# 1. Start fresh from main
git checkout main && git pull origin main
# 2. Create feature branch
git checkout -b feature/my-feature
# 3. Work and commit
git add . && git commit -m "feat: my feature"
# 4. Push + open PR
git push -u origin feature/my-feature
# 5. After PR merged, clean up
git checkout main && git pull
git branch -d feature/my-feature
git push origin --delete feature/my-feature
```

 HOTFIX WORKFLOW

```
git checkout -b hotfix/critical-bug main
git commit -am "fix: resolve critical bug"
git checkout main
git merge --no-ff hotfix/critical-bug
git tag -a v1.0.1 -m "Hotfix v1.0.1"
git push origin main --tags
# Back-merge into develop
git checkout develop && git merge hotfix/critical-bug
git branch -d hotfix/critical-bug
```

 FORK & SYNC UPSTREAM

```
# One-time: add upstream remote
git remote add upstream <original-repo-url>
# Sync fork with upstream
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

 MOST-USED COMMANDS AT A GLANCE

<code>git init</code> * New repo	<code>git clone <url></code> * Clone repo	<code>git add .</code> * Stage all	<code>git add -p</code> * Stage hunks
<code>git commit -m</code> * Commit	<code>git commit --amend</code> * Edit last commit	<code>git switch -c </code> * New branch	<code>git branch -d </code> * Delete branch
<code>git merge --no-ff</code> * Merge with commit	<code>git rebase -i HEAD~3</code> * Interactive rebase	<code>git push -u origin </code> * Push + track	<code>git fetch --all --prune</code> * Sync remotes
<code>git log --oneline --graph</code> * Visual log	<code>git diff --staged</code> * Review staged	<code>git reset --soft HEAD~1</code> * Undo last commit	<code>git revert <hash></code> * Safe undo
<code>git stash push -m</code> * Stash work	<code>git stash pop</code> * Restore stash	<code>git cherry-pick <hash></code> * Copy a commit	<code>git reflog</code> * Recovery history

REF / Key Comparisons

 GIT RESET MODES COMPARED

MODE	COMMIT	INDEX (STAGED)	WORKING DIR	USE WHEN
<code>--soft</code>	Moves back	Kept staged	Unchanged	Re-commit with changes
<code>--mixed</code>	Moves back	Unstaged	Unchanged	Default; re-add & commit
<code>--hard</code>	Moves back	Cleared	LOST	Nuclear option; irreversible

 FETCH VS PULL VS CLONE

COMMAND	DOWNLOADS	MERGES	CREATES REPO
<code>git fetch</code>	✓ remote changes	✗	✗
<code>git pull</code>	✓ remote changes	✓ auto-merges	✗
<code>git clone</code>	✓ entire repo	✓ initial	✓

 MERGE VS REBASE VS SQUASH

STRATEGY	HISTORY	SAFE ON SHARED	BEST FOR
<code>merge --no-ff</code>	Preserves all	✓ Yes	Feature integration to main
<code>rebase</code>	Rewrites	✗ No	Clean local history
<code>merge --squash</code>	Collapses	✓ Yes	Clean main with WIP branches

 HEAD NOTATION REFERENCE

HEAD		ORIG_HEAD	
HEAD	Current commit	FETCH_HEAD	Pre-merge/reset HEAD
HEAD~1	1 commit back	MERGE_HEAD	Last fetched ref
HEAD~3	3 commits back		Incoming merge commit
HEAD^	First parent	@{upstream}	Tracking branch
HEAD^2	Second parent (merge)	abc1234	Any short/full hash